

# Sjednocení dvou posloupností čísel

## Dokumentace k programu

Radek Modrůczki

19. února 2026

### Abstrakt

Program řeší úlohu nalezení sjednocení dvou celočíselných posloupností. Namísto využití vestavěných množinových operací a kontejnerů typu `set` je algoritmus postaven na setřídění spojených dat a následné eliminaci duplicit v jednom průchodu. Výsledná aplikace nabízí matematický výsledek a vhled do třídícího a vyhledávacího mechanismu.

## Obsah

|          |                                                     |          |
|----------|-----------------------------------------------------|----------|
| <b>1</b> | <b>Přesné zadání</b>                                | <b>2</b> |
| <b>2</b> | <b>Zvolený algoritmus</b>                           | <b>2</b> |
| <b>3</b> | <b>Diskuse výběru algoritmu</b>                     | <b>2</b> |
| <b>4</b> | <b>Program</b>                                      | <b>2</b> |
| 4.1      | Použité datové struktury . . . . .                  | 2        |
| 4.2      | Hlavní podprogramy . . . . .                        | 2        |
| 4.3      | Tok řízení . . . . .                                | 3        |
| <b>5</b> | <b>Alternativní programová řešení</b>               | <b>3</b> |
| 5.1      | Využití standardních množinových typů . . . . .     | 3        |
| 5.2      | Strategie Sort & Scan . . . . .                     | 3        |
| <b>6</b> | <b>Reprezentace vstupních dat a jejich příprava</b> | <b>3</b> |
| 6.1      | Reprezentace vstupních dat . . . . .                | 3        |
| 6.2      | Příprava vstupních dat . . . . .                    | 3        |
| 6.3      | Reprezentace výstupních dat . . . . .               | 4        |
| 6.4      | Interpretace výstupních dat . . . . .               | 4        |
| <b>7</b> | <b>Průběh práce</b>                                 | <b>4</b> |
| <b>8</b> | <b>Možná vylepšení</b>                              | <b>4</b> |

# 1 Přesné zadání

Na vstupu jsou dvě neuspořádané posloupnosti s různým počtem celočíselných prvků. Napište funkci, která nalezne jejich sjednocení, tj. unikátní prvky v obou posloupnostech, vytvoří z nich novou posloupnost a vytiskne ji. Obě posloupnosti bude nutné setřídít. Pro nalezení duplicit nepoužívejte kontejnery typu `set` ani další knihovny podporující vestavěné množinové operace.

## 2 Zvolený algoritmus

Pro nalezení sjednocení dvou neuspořádaných posloupností byl zvolen algoritmus, který obě posloupnosti nejprve samostatně seřadí, odstraní z nich duplicity a následně sloučí pomocí dvou ukazatelů.

Pro seřazení je využitý algoritmus Bubble Sort. Algoritmus projde každé seřazené pole a filtruje duplicity tak, že do nového pole přidává pouze prvky, které se nerovnají následujícímu číslu. Samotné sjednocení pak probíhá procházením obou vyčištěných posloupností. Vždy se porovnají aktuální prvky, menší se přidá do výsledku a ukazatel se posune. Takto vzniká nová, seřazená posloupnost.

## 3 Diskuse výběru algoritmu

Při návrhu programu byl zvažován naivní přístup (Brute Force), který by pro každý prvek ověřoval jeho výskyt ve výsledném poli prostým lineárním vyhledáváním. Tato varianta byla zavrhnuta z důvodu neefektivní časové složitosti, která by při větším objemu dat způsobila zásadní zpomalení aplikace.

Řešení pomocí hashovacích tabulek nebylo možné implementovat vzhledem k zadání aplikace. Zvolená metoda využívající předběžné setřídění se tedy jeví jako optimální řešení. Snižuje výpočetní náročnost a zároveň plní požadavek na uspořádaný výstup, čímž eliminuje nutnost volat třídící funkci v separátním kroku.

## 4 Program

Aplikace je implementována v jazyce Python s důrazem na objektově orientovaný přístup. Je striktně odlišena správa dat od samotné algoritmické logiky sjednocení.

### 4.1 Použité datové struktury

Pro uložení vstupních i výstupních dat jsou využity dynamické seznamy (typ `list`). Data jsou ukládána jako celá čísla, převod textového formátu probíhá při načítání.

### 4.2 Hlavní podprogramy

Veškerá algoritmická logika je rozdělena do následujících funkcí:

- `bubble_sort(arr)` – vlastní implementace řadícího algoritmu, porovnává sousední prvky a prohazuje je, pokud jsou ve špatném pořadí.
- `unique(arr)` – projde seřazené pole a odstraní z něj duplicity tak, že zanechá pouze prvky, které se liší od následujícího prvku.

- `get_union(a, b)` – hlavní výpočetní funkce, která přijímá dvě pole, seřadí je, očistí je od duplicit a pomocí předchozích funkcí je slučuje do konečného výsledku.

### 4.3 Tok řízení

Program se nejprve táže na název souboru. Následně ošetřuje případné chyby, načítá obě posloupnosti a převádí je na číselné seznamy. Ty následně předá funkci `get_union()` a hotové sjednocení naformátuje pro výpis do konzole.

## 5 Alternativní programová řešení

Při vytváření aplikace bylo zvažováno několik odlišných přístupů k implementaci, níže jsou uvedeny varianty, které byly v procesu zamítnuty a co k zamítnutí vedlo.

### 5.1 Využití standardních množinových typů

Toto řešení se jevilo jako nej přirozenější způsob konverze obou seznamů na datový typ `set` a následné použití operátoru sjednocení. Toto by řešení extrémně zjednodušilo, kdy kód by mohl být i v rozsahu několika řádků. Bylo explicitně zakázáno v zadání.

### 5.2 Strategie Sort & Scan

Původně bylo zamýšleno obě vstupní posloupnosti spojit do velkého seznamu, ten seřadit a z něj odstranit duplicity. Tato metoda byla zavrhnuta ve prospěch merge strategie. Paralelní slučování dvou uspořádaných množin je efektivnější a minimalizuje počet operací při spojování.

## 6 Reprezentace vstupních dat a jejich příprava

### 6.1 Reprezentace vstupních dat

Program pracuje výhradně s obyčejným textem. Aby aplikace správně fungovala a nebyla ukončena některou z možných chyb je nutné dodržet určitá pravidla pro formátování.

Program očekává jako vstup obyčejný textový soubor, který musí obsahovat přesně dva řádky, které určují dvě vstupní množiny. Čísla ve vstupním souboru musí být oddělena mezerou a musí být pouze celá čísla. Není možné používat čárky, středníky ani tabulátory pro oddělení. Text také nesmí obsahovat žádná písmena ani speciální znaky.

### 6.2 Příprava vstupních dat

Pro vytvoření vstupních dat je ideální obyčejný textový editor, jako např. Poznámkový blok ve Windows. Je nutné, aby po vytvoření textového souboru byl uložen do stejné složky jako samotný program, aby jej aplikace mohla najít a nedošlo k chybě.

Obsah souboru musí tvořit přesně dva řádky, na prvním řádku první posloupnost a na druhý řádek druhá posloupnost. Jednotlivá čísla od sebe musí být od sebe oddělena mezerou. V souboru se nesmí vyskytovat žádné jiné oddělovače, písmena ani prázdné řádky navíc. V případě porušení těchto pravidel program nebude fungovat.

Reprezentace výstupních dat a jejich interpretace

### 6.3 Reprezentace výstupních dat

Výsledky programu se vypisují formou textových zpráv přímo na obrazovku v příkazovém řádku, hned pod příkaz, kterým uživatel program spustil. Výstup je strukturován do tří oddělených bloků pro lepší čitelnost.

První řádek informuje o počtu čísel, která byla načtena ze vstupního souboru. Následuje výpis výsledné seřazené posloupnosti, kde jsou čísla oddělena čárkou. Výpis je ukončen informací na posledním řádku, která uvádí počet nalezených unikátních prvků.

### 6.4 Interpretace výstupních dat

První řádek ve výstupu slouží jako kontrola správného čtení vstupního souboru. V případě nulové hodnoty je vstupní soubor prázdný, nebo chybně formátovaný a program jej nemohl zpracovat.

Hlavní část výstupu představuje hledané sjednocení. Je výsledkem spojení obou vstupních řádků, program odstranil všechny duplicity a čísla seřadil od nejmenšího po největší. Počet prvků značí délku výsledné posloupnosti a umožňuje jednoduše zjistit počet unikátních prvků v obou řádcích.

## 7 Průběh práce

Na začátku práce bylo potřeba se seznámit s hlavními omezeními zadání, které zakazuje používání vestavěných množin. Následně bylo třeba se zamyslet nad algoritmickou podstatou problému. Zvažováno bylo naivní porovnání každého prvku s každým, což bylo pro vysokou časovou náročnost zamítnuto. Bylo poté zvoleno využití vlastností seřazeného pole, kde se duplicity přiřadí k sobě, což umožňuje efektivnější řešení problému.

Největší výzvou bylo správné ošetření vstupních dat. Problematické byly vstupní soubory s mezerami navíc nebo s prázdnými řádky. Bylo třeba zajištění robustnějšího načítání souborů. Správnost řešení byla ověřena na náhodně ručně vytvořených souborech, ve kterých byly různé počty duplicit, aby bylo ověřeno, zda vše funguje bezchybně.

## 8 Možná vylepšení

Program vyhovuje požadavkům zadání, avšak existuje prostor pro vylepšení a zdokonalení. Bylo by možné rozšířit aplikaci o podporu jiných datových typů, například textových řetězců nebo desetinných čísel, což by vyžadovalo úpravu parsovací logiky.

## Reference

- [1] PILGRIM, Mark. *Ponořme se do Pythonu 3* [online]. Praha: CZ.NIC, 2010. Dostupné z: <http://diveintopython3.py.cz/index.html>
- [2] PYTHON SOFTWARE FOUNDATION. *Python 3.12 Documentation* [online]. Dostupné z: <https://docs.python.org/3/>